Quantization for Reasoning Preservation in Small LLMs

Roshan Iruku
Central Peel Secondary School
roshaniruku@gmail.com

Gurshaan Dhillon
North Park Secondary School
gurshaan.dhillon10@gmail.com

Abstract

Large Language Models (LLMs) have demonstrated exceptional capabilities in textual inference, but their deployment on edge devices is limited to high computation and memory overhead. To address this, we provide LLM-QRP (**Q**uantization for **R**easoning **P**reservation), a layer-aware mixed-precision quantization framework which is designed to reduce a model's size while preserving its reasoning performance. Unlike uniform post-training quantization methods, LLM-QRP identifies and preserves critical "thinking "layers by analyzing the model's layer-wise logit evolution and entropy dynamics across transformer blocks. These signals are combined into a unified reasoning importance score, ranking layers according to their inference contribution. We then apply an empirical mixed-precision strategy that allocates higher precision to high-scoring layers and lower precision to redundant layers, optimizing the accuracy to compression trade-off. Experiments on multiple small models were trained, and evaluation on GSM8K and TruthfulQA shows that LLM-QRP significantly reduces VRAM usage while maintaining almost near-baseline reasoning performance, outperforming standard uniform 4-bit quantization. Our results demonstrate that layer-wise quantization guided by internal reasoning signals can enable deployment of LLMs on resource-constrained hardware without significant degradation in a model's reasoning ability.

1 Introduction

Large Language Models (LLMs) have qualitatively transformed various domains of artificial intelligence. A core advantage of local LLMs is the benefits of on-device use. It serves as an alternative pathway for LLMs to operate effectively offline, without intermediate delays from API or cloud calls, and remains beneficial for real-time, offline applications involving autonomous vehicles, financial processing, and applications requiring a fine-tuned model. However, as a wise man once said, "With great model size comes a great compute cost." For instance, Qwen3.5, containing 0.8B parameters at FP16, has a memory footprint of 1.8 GB, which can adequately run inference on edge devices [1, 14].

In post-training-quantization (PTQ), multi-step reasoning tasks are compromised first, leading to degradation in the model's ability to reason and think [17]. Quantization-aware training suffers from high infrastructure costs, making compute and model training quite expensive. Recent advancements in quantization provided by GPTQ and LLM-AWQ do provide mixed-precision quantization; LLM-AWQ optimizes for which weights to preserve instead of identifying layers that actually perform reasoning [18].

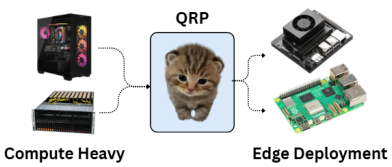

Figure 1: Quantization Overview

Preprint. Under review.

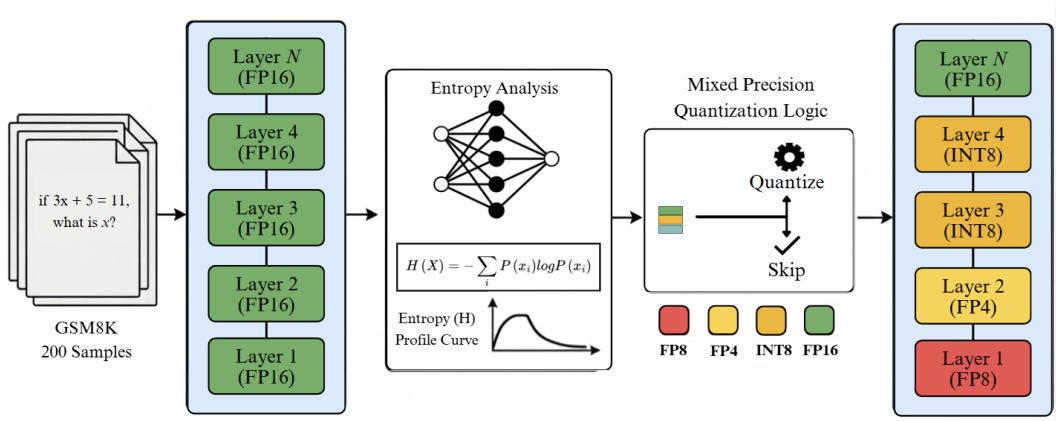

Figure 2: Illustration of our LLM-QRP workflow.

To address these challenges, we propose LLM-QRP (Quantization for Reasoning Preservation), a layer-aware quantization framework that explicitly targets the preservation of reasoning capability in small LLMs. To begin with, LLM-QRP identifies and prioritizes "thinking layers" based on their contribution to the model's final predictions during inference [1]. LLM-QRP leverages two primary metrics. First, we introduce SLED (self-logits evolution decoding), which tracks the evolution of token logits across layers and quantitatively measures latent knowledge distribution during pre-phase against the model's output distribution [28]. This metric is assisted by a submetric using the Kullback–Leibler (KL) divergence between adjacent layers and the final layers. Second, we incorporate layer-wise entropy analysis, which measures the "sharpness" of the token distribution at each layer and reflects the model's confidence across intermediate layers.

These signals are aggregated into a single-layer normalized score that ranks layers according to their reasoning contributions. Intuitively, layers with low KL divergence to the final distribution and significant entropy reduction are interpreted as "critical reasoning" layers, as they transfer latent information across intermediate layers with minimal information loss. To verify the accuracy of these "thinking layers," they were validated through a set of ablation studies. To synthesize these layer scores for each model, we empirically evaluate LLM-QRP on reasoning-intensive benchmarks, including GSM8k, using chain-of-thought prompting. Layer metrics for every prompt are aggregated into a unified score [5].

To determine quantization precision boundaries, we empirically evaluate mixed-precision configurations guided by the principle of assigning lower precision to low-scoring layers while preserving higher precision for high-scoring layers. We systematically test different layer-wise precision combinations and select thresholds based on performance on the GSM8K benchmark. Final benchmarks were conducted across four sub-500M parameter models, comparing full-precision, standard quantized, and LLM-QRP-guided quantized variants. We further benchmark against existing approaches such as AWQ [5].

- We propose LLMs-QRP as a layer-wise quantization method to adequately maintain a small LLM reasoning presentation while ensuring minimal loss in model performance.
- Combines KL divergence and entropy into a normalized score to rank layers by reasoning importance, validated through ablation studies and chain-of-thought benchmarks (e.g., GSM8k).

[1] In this case, A layer is considered a thinking layer if it contributes significantly to the alignment between intermediate and final output distributions, measured through Self Logits Evolution and final output distribution change through layer sensitivity under layer ablation

Algorithm 1: LLM-QRP: Quantization via Reasoning Prioritization

Input : Model $\mathcal{M}$ with L layers, calibration set $\mathcal{P}$, accuracy floor $\alpha_{\min}$
Output Quantized model $\mathcal{M}^*$, benchmark results $\mathcal{R}$
:

for *each prompt* $p_k \in \mathcal{P}$ **do**
  Generate logits $\{\ell_0, \ldots, \ell_L\}$ and distributions $\{P_0, \ldots, P_L\}$;
  $\tilde{S}_i^{(k)} \leftarrow \textsc{MinMaxScale}(\mathit{SLED}(L_i))$ for all layers i;
  $\tilde{K}_i^{(k)} \leftarrow \textsc{MinMaxScale}(D_{KL}(P_{i-1} \| P_i))$ for all layers i;
  $\tilde{H}_i^{(k)} \leftarrow \textsc{MinMaxScale}(|H(L_i) - H_{\text{base}}|)$ for all layers i;
  $T_i^{(k)} \leftarrow \frac{1}{2}\left(\frac{\tilde{S}_i^{(k)} + \tilde{K}_i^{(k)}}{2} + \tilde{H}_i^{(k)}\right)$
$T_i \leftarrow \frac{1}{N}\sum_k T_i^{(k)}$; $\quad \mathbf{L}_{\text{sorted}} \leftarrow \textsc{SortAscending}(\{(i, T_i)\})$;
Perform progressive ablation on $\mathbf{L}_{\text{sorted}}$ to validate reasoning importance;
for *each pair* (p_4, p_8) *with* $0 \le p_4 \le p_8 \le 100$ *in steps of* 10% **do**
  Assign bottom $p_4\%$ layers $\rightarrow$ FP4, next $(p_8 - p_4)\% \rightarrow$ INT8, rest $\rightarrow$ BF16;
  $\eta \leftarrow \Delta\text{Size} \,/\, \Delta\text{Accuracy}$
$\mathit{config}^* \leftarrow \arg\max \eta \quad \text{s.t. } a_{\text{q}} \ge \alpha_{\min} \cdot a_{\text{base}}; \quad \mathcal{M}^* \leftarrow \textsc{Quantize}(\mathcal{M}, \mathit{config}^*)$;
Evaluate $\mathcal{M}^*$ on benchmark set $\mathcal{D}$ and return results $\mathcal{R}$;

2 Related Works

LLM-AWQ is a quantization framework focusing on preserving a model's salient weights. LLM-AWQ uses Post-Training Quantization (PTQ) to compress large language models efficiently without the immense cost of retraining. We follow the same idea; however, we primarily concentrate on a layer-based method instead of utilizing the model's salient weights. Furthermore, LLM-AWQ does not assess reasoning degradation empirically for each model, allowing a generalized quantized approach which may not completely fit a given model's specific characteristics or architecture [18].

3 Quantization for Reasoning Preservation in Small LLMs

We observe that not all layers contribute equally to the model's final response, and there exists an optimal distribution of quantized layers such that the model retains most of its efficacy whilst maintaining a significantly smaller memory footprint. The intuition behind this is to find the "thinking layers. [15]"

3.1 Logits Evolution Across Transformer Layers

Modern large language models are architecturally composed of N model layers, with a vocabulary space defined as $V = [v_1, v_2 ... v_{n-1}]$. The first metric compares the current KL divergence between the current defined as L_i and L_{i+1} by projecting high-dimensional hidden states back to the vocabulary space and converting them into logit values using the LM head (W). Then a softmax is applied to these logits, converting them into probability distributions.

$$\ell_i = W \cdot h + b \tag{1}$$

To convert these into a probability distribution, we apply the Softmax function.

$$P_i(v) = \frac{\exp(\ell_i)}{\sum_{j=i}^{|V|} \exp(\ell_i)} \tag{2}$$

To measure the "thinking" occurring between adjacent layers, we calculate the KL Divergence:

$$D_{KL}(\mathcal{P}_i||\mathcal{P}_{i+1}) = \sum_{v \in V} P_i(v) \log\left(\frac{P_i(v)}{P_{i+1}(v)}\right) \tag{3}$$

After applying a standard KL divergence analysis, we can use SLED-specific methods, which compute a weighted average of these intermediate layers, with layers that contain more "accurate" signals given greater influence on the final output, more accurately representing knowledge transfer between latent layers. Here, the core intuition is that a "Thinking Layer" is not just one that changes, but one whose change aligns semantically with the final output. First, we define a set of candidate premature layers at a fixed stride $\mathcal{L}_{cand} = \{L_1, L_5 ... L_N\}$, as choosing each layer is too computationally expensive. We can then project both candidate layers and final mature layers into logits values [28].

$$\ell_i = W\,\mathrm{Norm}(h_i), \quad \text{for } L_i \in \mathcal{L}_{\text{cand}} \tag{4}$$

Let $\ell_M = W \cdot Norm(h_N)$ be a candidate mask on a relative log-probability threshold (e.g., 0.1) from the mature output, eliminating any vocabulary noise. For each candidate layer, we define the Evolution Gradient relative to the top-k tokens using a one-hot token method and the Logit Divergence

$$g_{(i,t)} = softmax(\ell_i) - \mathbf{1}_t, \quad \delta_i = log\,softmax(\ell_i) - log\,softmax(\ell_M) \tag{5}$$

Finally, the SLED score is the Cosine Similarity between the masked gradient and masked divergence,

$$SLED(L_i) = \cos\_sim(g_i^M, \delta_i^M) \tag{6}$$

With the final results providing us a SLED score between $[0, 1]$ with an emphasis on KL divergence, "thinking" through the amount of layer disagreement between adjacent layers.

3.2 Layer-wise Entropy Profiling

The concept of entropy is applied to gauge an LLM's uncertainty as it moves through the residual stream, thereby defining its probabilistic convergence. While a model converts tokens to natural language at the language modelling head, a probability distribution is updated at every layer; each intermediate distribution can be calculated by passing the layer's state through a logit lens [23]. This involves forcing the middle layer's hidden state through the logit lens projection, depicting the model's uncertainty at layer N as a projection of latent representations in probability space. Ideally, the layers transition from high-entropy (semantic exploration) to low-entropy (probabilistic convergence) as the model converges upon a thought; each layer contributes a marginal KL-divergence towards the final logit distribution [29].

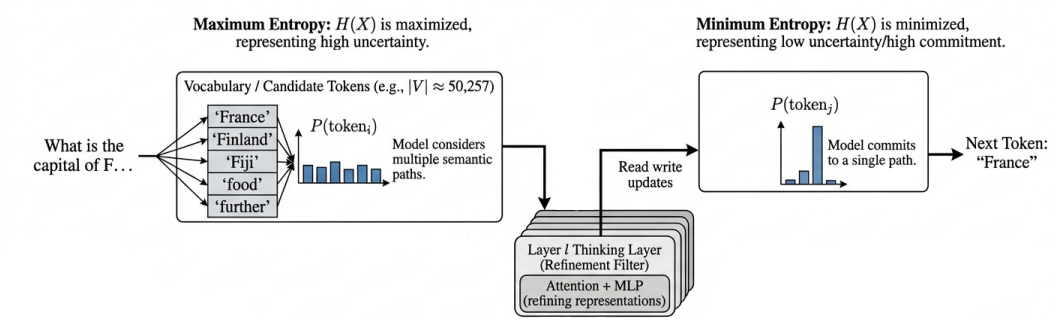

Figure 3: Probabilistic convergence through the residual stream. High-entropy exploration in early layers transitions to low-entropy convergence in final reasoning layers.

Rather than measuring raw entropy, this paper studies how sensitive a model's final entropy is to changes in precision from each layer, or Structural Entropy Sensitivity. The layers with the highest quantization sensitivity are the most vital to the inference process; the model's performance will fall when they are lowered in precision as compared to less sensitive layers.

The ΔH metric is implemented algorithmically, beginning with a forward pass in BFloat16 (BF16) precision to determine the output's original entropy and distribution. Subsequently, a layer-wise quantization sensitivity test is conducted: for each transformer block i within the multi-head attention and feed-forward network projection, a 4-bit quantization is injected. The Shannon entropy of this modified distribution is then computed as the uncertainty of a model's output distribution P over the vocabulary V.

$$H(P) = -\sum_{v \in V} P(v) \log(P(v)) \tag{7}$$

After computing this value, the Structural Entropy Sensitivity is evaluated as the absolute deviation in entropy relative to the baseline. High ΔH_i values indicate layer importance and information bottlenecks. Finally, these metrics culminate in developing the reasoning importance rank used to apply mixed-precision.

$$\Delta H_i = |H(P_{\text{base}}) - H(P_{\text{q},i})| \tag{8}$$

3.3 Multisignal Normalization

$$\text{Norm}_{\text{layer}}(x) = \frac{x - \min\left(S_{\text{layer}}, H_{\text{layer}}\right)}{\max\left(S_{\text{layer}}, E_{\text{layer}}\right) - \min\left(S_{\text{layer}}, E_{\text{layer}}\right)} \tag{9}$$

Once normalized values are calculated, we empirically find "thinking" scores during inference by using the GSM8K dataset. 200 samples were chosen, layer data was calculated, and the layer data of each prompt was merged into a final mean score of all the total layer data for each prompt.

3.4 Mixed-Precision Quantization for Reasoning Preservation

We employ a Mixed-Precision Quantization (MPQ) framework to convert the reasoning importance scores achieved through layer ablation and SLED/entropy analyses into a tangible structure for model deployment [20, 11]. This strategy partitions model layers and allocates precision between them rather than quantizing models all in one precision [12]. While quantization is bound to reduce performance due to the nature of compressing model weights [10], mixed-precision allows for a greater model accuracy-to-size ratio by compressing only certain layers [24, 8]. Transformer models do not act as a monolithic structure but rather as a pipeline of stages [22], passing information and transforming it as it traverses the residual stream [15]. Some of these stages are more thinking-intensive, while others just relay the information with little change; a mixed-precision strategy works well to respect this downstream architecture [16].

After the reasoning importance scores are computed, they are used to rank each layer's contribution to the final output. The lowest layers on this ranking are then quantized appropriately, and their precision is instead allocated towards the higher importance layers [25, 18]. Once layers are in a ranking, the bottom percentile is assigned to FP4, the middle percentile to INT8, and the remainder preserved in BF16, or full precision [6, 7]. This algorithm runs to find a Pareto Front for quantization [9], where the bit-rate has a significant drop while retaining accuracy above a defined floor, α_{min} [5, 27].

However, each model's layers are composed under a different architecture, and the same algorithm for precision allocation will not be applicable universally [1, 2]. For this reason, we searched for an optimal configuration within each model by performing a hyperparameter sweep [4]. The test results validated this theory: while 8-bit quantization retained the baseline accuracy [17], 4-bit quantization indicates a massive fall in performance, or an increase in quantization noise [21, 13]. The mixed-precision approach, however, balances between these two, causing a slight drop in model performance but at a strong ratio to its weight compression [26].

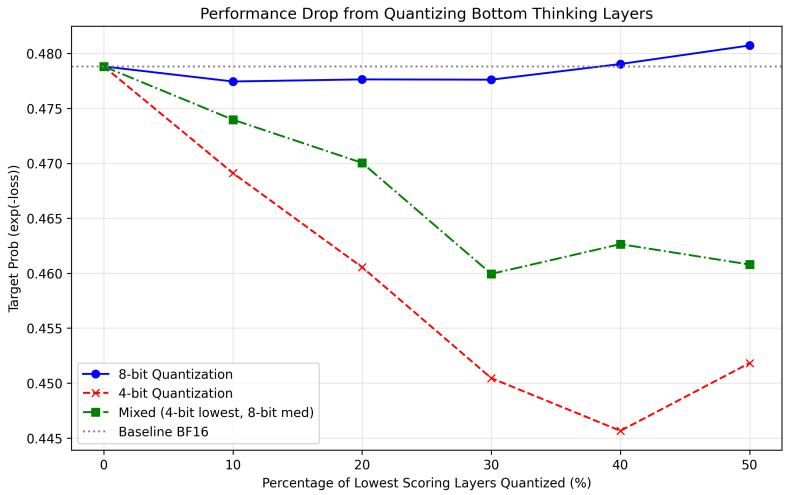

Figure 4: Empirical performance degradation under varying quantization protocols. The QRP strategy (Mixed) significantly outperforms uniform 4-bit baselines by selectively preserving high-sensitivity thinking layers.

During the quantization phase of the algorithm, a GPTQ-based engine is employed to perform weight compression. QRP acts as the strategy to allocate precision, while GPTQ allows for Hessian-based error compensation to minimize reconstruction loss. As a result of this synergy, we can achieve high proportions of size decrease while maintaining overall model accuracy [3].

Running on Qwen2.5-0.5B, baseline BF16 weights would take ~715.7 MB of VRAM, while our QRP protocol decreased this value to ~372.7 MB, or a 47.9% reduction in size while retaining near lossless accuracy on GSM8K. This compression lowers the computation barrier for edge devices, allowing reasoning to be accessible for IoT devices and microchips.

4 Experiments and Benchmark

As a novel layer-wise quantization method, we benchmarked the accuracy of our quantized model through GSM8K and TruthfulQA across a variety of sub-500 M parameter models from various families, such as (Qwen, Granite, LLAMA). We also conducted empirical experiments to determine the sensitivity of quantization and proof of layer-wise thinking [19, 5].

4.1 Verification of ("Thinking") Layers

Now, we wanted to verify whether layers assigned higher "thinking scores" are truly more important for reasoning than those with lower scores. To test this, we utilized a layer ablation-based approach, where specific layers are systematically removed from the model, and the impact on the performance is measured. The ablation process constructs a modified version of the model by excluding selected layer indices and replacing the original layer set with this modified stack. After each trial, the model restores its layers to their original set.

Two parallel ablation strategies are conducted to evaluate the validity of the thinking scores. To begin with, layers belonging to the bottom 20% of thinking scores are progressively removed under the assumption that these layers contribute less to reasoning. In the second layer, the layers belonging to the top 20% of thinking scores are removed, which are hypothesized to be critical to the model's thinking acumen.

Performance is quantified using exp(average loss), with lower loss corresponding to higher performance. If the thinking scores accurately depict the layer importance, removing 20% of the top layer should result in a greater performance decline than the bottom 20%.

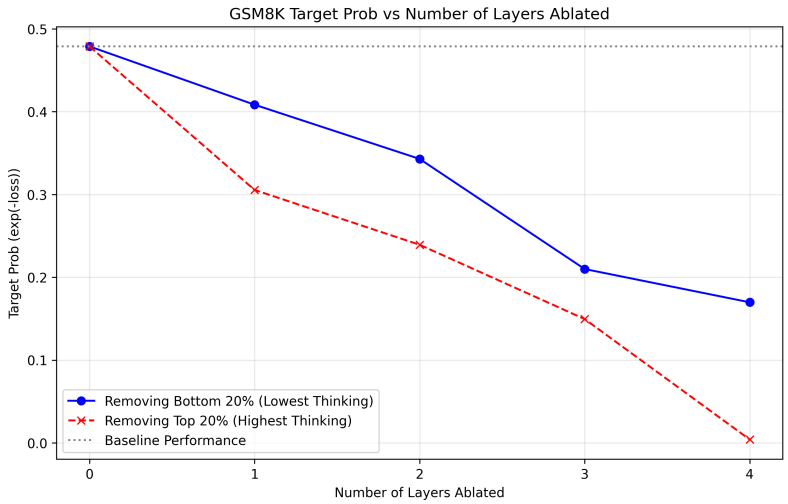

Figure 5: Performance drop measured after top and bottom 20% layers are sorted by thinking scores and ablation tested on Qwen2.5-0.5B.

Table 1: **Mixed-precision quantization performance across model families.** *VRAM Reduction* denotes memory savings relative to BF16. *Reasoning Retention* measures normalized performance relative to the BF16 baseline (values > 100% indicate improvement). *Efficiency Gain* reflects the relative increase in target prediction confidence compared to uniform 4-bit quantization.

Model	VRAM Reduction (%)	Reasoning Retention (%)	Efficiency Gain (%)
SmolLM2-135M	50.0	92.1	+91.3
Qwen2.5-0.5B	53.1	113.9	+68.2
LFM2.5-350M	56.3	85.2	+42.5
Average	**53.1**	**97.1**	**+67.3**

4.2 Experimental Setup

Benchmark Our method of quantization is evaluated on accuracy-based and multiple-choice datasets such as TruthfulQA, FACTOR, and GSM8K to assess the model's accuracy.

To provide a comprehensive evaluation of our QRP algorithm, we quantify model performance through VRAM reduction, reasoning retention, and efficiency gain. VRAM reduction is computed as the difference in memory footprint between baseline and QRP-quantized models. A higher VRAM footprint indicates difficulty deploying to edge devices, where computational power is limited. Reasoning retention represents the percentage of baseline performance preserved post-quantization, while efficiency gain is defined as a relative improvement in reasoning-per-megabyte compared to standard uniform quantization.

As indicated in Table 1, LLM-QRP yields a mean efficiency gain of over 67%, reinforcing how retaining precision on reasoning-dense layers can achieve a more optimal utility-to-size density.

Models We evaluate the performance of LLM-QRP on three sub-500M parameter models from diverse architectural families. The models considered are SmolLM2-135M, Qwen2.5-0.5B, and LFM2-350M. We compare against standard quantization baselines, including LLM-AWQ and other conventional post-training quantization techniques.

4.3 Evaluation on a Broad Range of LLM Benchmarks

LLM-QRP has achieved empirical results through testing on multiple LLM architectures, including Qwen and SmolLM. While uniform 4-bit quantization compresses weights, QRP proposes an algorithm

to find the optimal mixed-precision mapping, optimizing the quantization process. The GSM8K performance displayed substantial improvements compared to uniform 4-bit and baseline BF16 quantization. Ultimately, the combined layer reasoning importance statistic is a universal indicator across transformers, allowing for accurate compression despite structural variances.

4.4 Evaluation Across Diverse LLM Configurations

The objective of these tasks is to verify accuracy retention during quantization. Two datasets were employed to verify these results, GSM8K and TruthfulQA. While GSM8K tests logical reasoning and the ability to maintain probabilistic convergence, TruthfulQA tests knowledge and factual understanding. The results presented in Table 1 demonstrate that LLM-QRP consistently outperforms other methods in model accuracy during the quantization process across all evaluated metrics for our models, including SmolLM2-135M, Qwen2.5-0.5B, and LFM2-350M. Importantly, LLM-QRP retained nearly 90% accuracy on GSM8K, all while reducing VRAM utilization by approximately 50% .

5 Limitations

The primary limitation of the provided quantization method is the high computational expense for larger models. Most of the experiments in this paper were performed on consumer-grade graphics cards, specifically the NVIDIA GeForce RTX 4070, which may neglect performance or scalability in training larger models or on different hardware. Additionally, LLM-QRP's evaluation relies on a limited set of datasets, which may not adequately capture edge cases or provide sufficient diversity in sample distribution. For instance, there is strong evidence that GSM8K test examples have leaked into the training data of many Large Language Models (LLMs), resulting in an optimistic bias in performance. In future trials, an "unseen" dataset can be introduced to calibrate the model.

6 Conclusion

In this paper, we propose LLM-QRP, a layer-based quantization framework for the preservation of reasoning in small models for use on edge computing. Through the compilation of SLED (Self-Logit Evolution Decoding) and structural entropy sensitivity (ΔH), we discovered the layer subsets most vital to reasoning in low-parameter LLMs, verifying this with an ablation study and benchmarking. Ultimately, our empirical results over multiple LLM model architectures proved that certain layers are relied upon as critical bottlenecks, and that quantizing redundant layers while retaining these bottlenecks can achieve an almost 2x improvement in VRAM use. LLM-QRP provides the blueprint for a strategy to improve the deployment of reasoning-capable models on edge hardware and limited-resource environments, creating a buffer between strong model intelligence and hardware requirements.

References

[1] URL `https://ollama.com/huihui_ai/qwen3.5-abliterated:0.8B-fp16`.

[2] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025. URL `https://arxiv.org/abs/2502.02737`.

[3] AutoGPTQ. Autogptq/autogptq: An easy-to-use llms quantization package with user-friendly apis, based on gptq algorithm. URL `https://github.com/AutoGPTQ/AutoGPTQ`.

[4] Jerry Chee, Yaohui Cai, Christopher De Sa, and Christopher R. Ré. Quip: 2-bit quantization of large language models with incoherence processing, 2024. URL `https://arxiv.org/abs/2307.13304`.

[5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

[6] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022. URL https://arxiv.org/abs/2208.07339.

[7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. URL https://arxiv.org/abs/2305.14314.

[8] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. Spqr: A sparse-quantized representation for near-lossless llm weight compression, 2023. URL https://arxiv.org/abs/2306.03078.

[9] Xuyang Fang et al. Slim-llm: Salience-driven mixed-precision quantization for large language models, 2024. URL https://arxiv.org/abs/2405.14159.

[10] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2023. URL https://arxiv.org/abs/2210.17323.

[11] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference, 2021. URL https://arxiv.org/abs/2103.13630.

[12] Zizheng Han et al. Llm-mq: Mixed-precision quantization for efficient llm deployment, 2024. URL https://arxiv.org/abs/2406.10238.

[13] Coleman Hooper et al. Spinquant: Llm quantization with vectorized weight rotation, 2024. URL https://arxiv.org/abs/2405.16437.

[14] Xinyi Hou, Yanjie Zhao, and Haoyu Wang. Llm applications: Current paradigms and the next frontier, 2025. URL https://arxiv.org/abs/2503.04596.

[15] Ferdinand Kapl, Emmanouil Angelis, Tobias Höppe, Kaitlin Maile, Johannes von Oswald, Nino Scherrer, and Stefan Bauer. Do depth-grown models overcome the curse of depth? an in-depth analysis, 2025. URL https://arxiv.org/abs/2512.08819.

[16] Yukun Li et al. Mxsens: Sensitivity-aware mixed-precision quantization for efficient llm inference, 2024. URL https://arxiv.org/abs/2408.01234.

[17] Zhen Li, Yupeng Su, Songmiao Wang, Runming Yang, Congkai Xie, Aofan Liu, Ming Li, Jiannong Cao, Yuan Xie, Ngai Wong, and Hongxia Yang. Quantization meets reasoning: Exploring and mitigating degradation of low-bit llms in mathematical reasoning, 2026. URL https://arxiv.org/abs/2505.11574.

[18] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration, 2024. URL https://arxiv.org/abs/2306.00978.

[19] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods, 2022. URL https://arxiv.org/abs/2109.07958.

[20] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Maziar van Baak, and Tijmen Blankevoort. A white paper on neural network quantization, 2021. URL https://arxiv.org/abs/2106.08295.

[21] Albert Tseng et al. Quip-anit: 1-bit weight quantization of llms with randomized hadamard transforms, 2024. URL https://arxiv.org/abs/2402.04349.

[22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL https://arxiv.org/abs/1706.03762.

[23] Zhenyu Wang. Logitlens4llms: Extending logit lens analysis to modern large language models, 2025. URL https://arxiv.org/abs/2503.11667.

[24] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Zheyuan Jeffrey Yu, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models, 2023. URL https://arxiv.org/abs/2211.10438.

[25] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Samyam Rajbhandari, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers, 2022. URL https://arxiv.org/abs/2206.01861.

[26] Vage Yatsenko et al. Extreme compression of large language models via additive quantization, 2024. URL https://arxiv.org/abs/2401.06118.

[27] Hugh Zhang, Jeff Da, Dean Lee, Vaughn Robinson, Catherine Wu, Will Song, Tiffany Zhao, Pranav Raja, Charlotte Zhuang, Dylan Slack, Qin Lyu, Sean Hendryx, Russell Kaplan, Michele Lunati, and Summer Yue. A careful examination of large language model performance on grade school arithmetic, 2024. URL https://arxiv.org/abs/2405.00332.

[28] Jianyi Zhang, Da-Cheng Juan, Cyrus Rashtchian, Chun-Sung Ferng, Heinrich Jiang, and Yiran Chen. Sled: Self logits evolution decoding for improving factuality in large language models, 2025. URL https://arxiv.org/abs/2411.02433.

[29] Xinghao Zhao. Entropy trajectory shape predicts llm reasoning reliability: A diagnostic study of uncertainty dynamics in chain-of-thought, 2026. URL https://arxiv.org/abs/2603.18940.